

人工智能课程设计——课程作业 2

搜索模块：机器人避障寻径问题

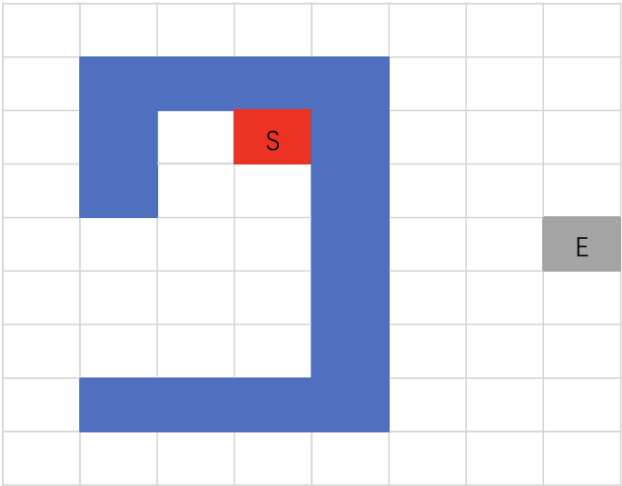
一、实验背景

机器人自主导航是智能机器人技术体系中的核心研究方向之一，避障寻径作为自主导航的关键环节，直接决定了机器人在非结构化、动态复杂环境中的作业能力与安全性。在实际应用场景中，无论是家庭服务机器人、仓储自动导引车（AGV），还是野外探测机器人，都需要具备在已知或未知环境中从起点自主规划出一条安全、高效路径的能力。如何平衡路径的最优性、搜索效率以及对环境的适应性，是路径规划算法研究的核心问题。

为系统探究不同路径规划算法的性能特点与适用场景，本实验以静态栅格地图下的机器人避障寻径为基础任务场景，设定固定起点（S）、终点（E）及不可通行的障碍区域，要求机器人从起点出发绕过障碍抵达终点。实验采用宽度优先搜索（BFS）与 A* 算法作为经典搜索方法的代表，通过可视化手段直观展示两种算法的搜索过程与最终路径，从算法原理、搜索策略、时间效率及空间消耗等多个维度进行对比分析。此外，实验进一步探讨了强化学习方法在该类问题中的可行性与改进空间，以期为不同工程需求下的路径规划算法选型提供参考依据。

二、实验内容

本实验在给定的静态栅格地图中实现机器人避障寻径任务。地图以矩阵形式表示，其中包含起点 S、终点 E 以及若干蓝色障碍区域，机器人需从 S 点出发，绕过障碍，抵达 E 点，并确保所走路径最短。



实验主要包括以下内容：

- 经典算法实现：**分别采用 A* 算法和 BFS 算法进行路径搜索，动态可视化展示搜索过程及最终路径，并对比两种算法的原理、流程及性能差异。
- 扩展算法探索：**自主设计更高难度的场景图，结合强化学习（如 Q-learning）或启发式算法（如遗传算法、人工势场法）进行路径规划，并尝试改进。

3. **补充对比分析：**引入 Dijkstra 等其他经典算法作为对比，分析各类算法的局限性及优化空间。

通过上述实验，旨在深入理解不同路径规划算法的特点，为实际应用中算法选型提供参考依据。

三、BFS 和 A*算法

算法原理：

广度优先搜索（Breadth-First Search, BFS）是一种经典的图遍历算法。其核心思想是**分层搜索**。

- **基本机制：**算法从起点 S 开始，优先访问其所有相邻的未访问节点。只有当当前层级的所有节点都被扩展完毕后，算法才会向下一层（即距离起点更远的一步）进行横向拓扑扩展。
- **数学本质：**由于栅格地图中各相邻格子之间的移动代价是均等的（均为 1 步），BFS 展开的搜索面在数学上等价于以起点为中心的拓扑同心圆。它是一种**盲目型搜索算法**，不具备任何关于终点位置的先验知识。
- **完备性与最优性：**在有限的静态栅格环境中，BFS 具有完备性（只要有路径就一定能找到）和最优性（第一次找到终点时，所经过的路径必然是最短路径）。

A* 算法是一种具有代表性的**启发式图搜索算法**（Heuristic Search）。它通过引入**估价函数**来引导搜索方向，避免了 BFS 的盲目扩散。

A* 算法的核心估价公式为：

$$f(n) = g(n) + h(n)$$

- **$g(n)$ （实际代价）：**从起点到当前节点 n 的实际移动步数。
- **$h(n)$ （启发式代价）：**从当前节点 n 到终点 E 的预估代价。本实验由于机器人只能在栅格中进行上下左右四个方向移动，故采用曼哈顿距离（Manhattan Distance）作为启发式函数，公式为：

$$h(n) = |x_n - x_E| + |y_n - y_E|$$

- **启发式引导机制：**A* 算法在每一步选择扩展节点时，都会优先选择 $f(n)$ 值最小的节点。这意味着它在考量“已经走了多远”的同时，重点预测了“哪个方向离终点最近”，从而使搜索波门强烈地向终点方向倾斜。

算法流程：

首先对于图进行编码，其中每个节点用一个坐标代表，可以到达的节点的值 0，障碍的值为 1。同时设置 start_pos 和 end_pos 表示起终点。

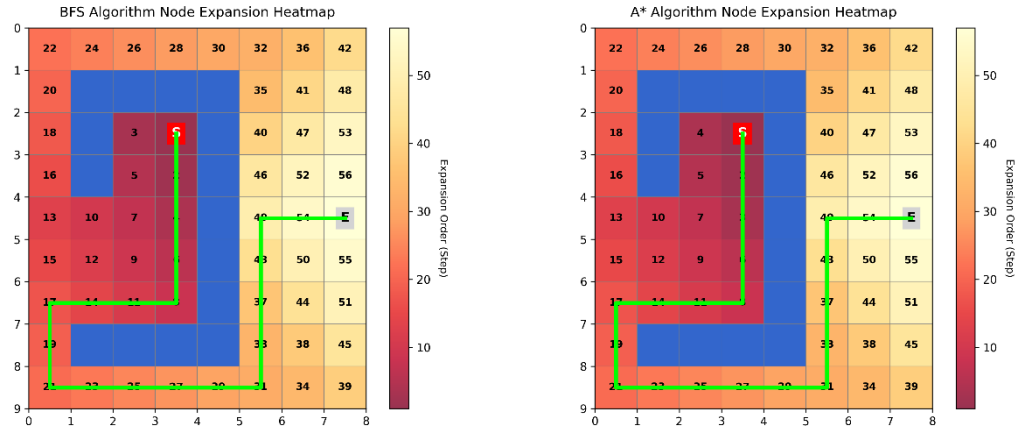
```
matrix = [  
    [0, 0, 0, 0, 0, 0, 0, 0],  
    [0, 1, 1, 1, 1, 0, 0, 0],  
    [0, 1, 0, 0, 1, 0, 0, 0],  
    [0, 1, 0, 0, 1, 0, 0, 0],  
    [0, 0, 0, 0, 1, 0, 0, 0],  
    [0, 0, 0, 0, 1, 0, 0, 0],  
    [0, 0, 0, 0, 1, 0, 0, 0],  
    [0, 1, 1, 1, 1, 0, 0, 0],  
    [0, 0, 0, 0, 0, 0, 0, 0]  
]
```

start_pos = (2, 3) # 对应红色 S 点
end_pos = (4, 7) # 对应灰色 E 点

两个算法的详细流程。

步骤	广度优先搜索 (BFS) 流程	A* 算法流程
1. 初始化	创建一个先进先出 (FIFO) 队列 queue，将起点 S 压入，并用哈希表 visited 记录访问轨迹。	创建一个基于最小堆的优先队列 open_set，计算起点的 $f(S) = h(S)$ ，将 (f, S) 压入；创建 g_score 表记录实际代价。
2. 循环迭代	若 queue 不为空，弹出队首节点作为当前节点 current。	若 open_set 不为空，弹出堆顶（即 $f(n)$ 最小）的节点作为 current。
3. 终点判定	若 current == end，则搜索成功，开始回溯路径。	若 current == end，则搜索成功，开始回溯路径。
4. 状态记录	将 current 写入扩展顺序表 expansion_order，计数器加 1。	将 current 写入扩展顺序表 expansion_order，计数器加 1。
5. 邻居扩展	遍历 current 上下左右合法且未访问的邻居： 1. 将其标记为已访问； 2. 记录其父节点； 3. 送入 queue 尾部。	遍历 current 上下左右合法的邻居，计算通过当前点到达该邻居的临时代价 tentative_g = g[current] + 1；若该代价小于已知代价，则更新 g_score 和 f_score，记录父节点，并将其送入 open_set。

可视化的结果：



算法结果差异：

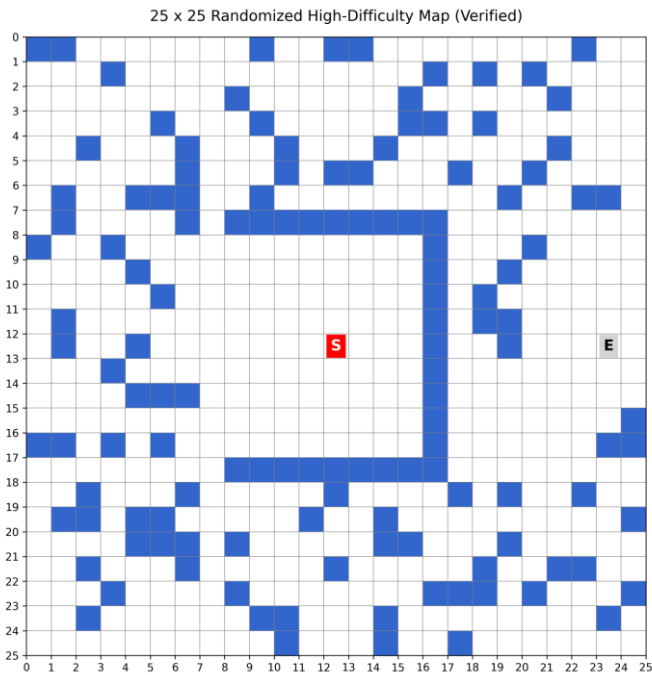
评估指标 (Metrics)	广度优先搜索 (BFS)	A* 算法 (曼哈顿距离)	性能对比与学术结论
最终路径长度 (Path Length)	20 步	20 步	双方均成功寻找到全局最优最短路径。
总扩展节点数 (Visited Nodes)	57 个	57 个	算法发生重合，A* 未能实现空间剪枝。
求解运行时间	0.1341 ms	0.3133 ms	BFS 速度快约 57.2%，

本组实验数据呈现出一个非常经典且具有学术研究价值的现象：在当前场景下，启发式搜索算法（A*）的各项指标并未压倒性胜过盲目搜索算法（BFS），反而产生了时间开销的增加。针对该现象，本报告从底层算法机理上进行如下深层解构：

1. “局部盲巷”导致的启发式失效 当前地图设计中，起点 S 处于一个由连续障碍物构成的巨大凹形死胡同（Dead End）内部。由于终点 E 位于其右侧，A* 算法在启发函数 $h(n)$ 的贪心引导下，会强烈优先扩展右侧靠近墙壁的节点。然而，右侧障碍物完全阻断了通行可能，算法必须原路折返至左侧唯一的狭窄出口。在突破该局部陷阱前，A* 算法不可避免地对陷阱内的所有合法网格进行了遍历。在有限的封闭空间内，其总扩展节点数（57 个）最终与盲目向外分层扩散的 BFS 算法完全一致。
2. 算法时间复杂度的常数项开销差异 当 A* 算法由于环境约束无法进行有效的空间剪枝时，其时间复杂度中的常数项开销便暴露出来。BFS 采用的先进先出（FIFO）队列在节点的压入与弹出操作上复杂度为 $O(1)$ ；而 A* 算法为了动态获取最优估价节点，必须维护一个基于最小堆的优先队列，其插入和弹出操作复杂度为 $O(\log N)$ ，且每一步都需要进行曼哈顿距离的代数运算。在扩展节点总数对等的前提下，A* 算法的计算冗余度显著高于 BFS，导致其耗时（0.3133 ms）反超 BFS（0.1341 ms）。

四、启发式算法与强化学习

（1）高难度场景图



该环境的构建基于以下核心设计思想：

1. **局部极小值物理陷阱：**在地图核心区域设计了一个开口向左、底部封闭向右的连续“U 型局部极小值口袋陷阱”。将起点 $S(12,12)$ 置于口袋的腹地深处，而将终点 $E(12,23)$ 部署在口袋外侧的最右端。起点与终点在几何上几乎处于同一水平线，直线距离极近。
2. **非结构化多干扰环境：**在口袋外围区域，引入了数学期望密度为 22% 的**高度离散随机障碍物（乱石堆带）**。这用于模拟无人机丛林穿梭、非结构化废墟搜救或行星表面探测等极端未知的真实控制场景，大幅度压缩了可行通路的容错率。
3. **全局可达性验证：**该场景在生成时，后台配置了基于连通性验证的广度优先搜索（BFS）过滤器，确保在外围高度随机化的干扰下，依然存在至少一条能完全绕出陷阱、通过乱石堆到达终点的“生命通道”。

（2）传统人工势场算法（APF）

传统人工势场法（Artificial Potential Field, APF）将机器人在环境中的运动抽象为虚拟物理势场下的质点运动。目标点产生引力场 U_{att} ，障碍物产生斥力场 U_{rep} ，合势场为：

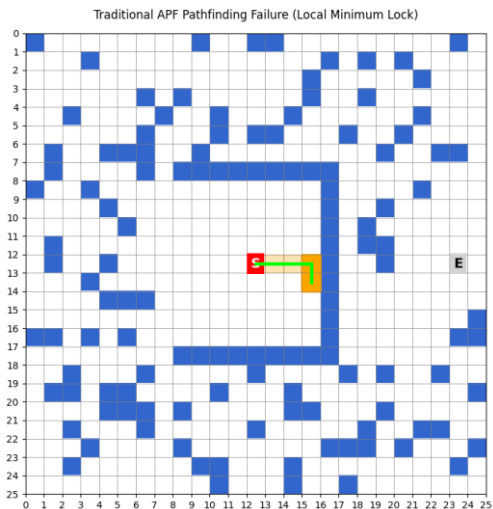
$$U_{total}(q) = U_{att}(q) + \sum U_{rep}(q)$$

机器人沿着总势场下降最快的负梯度方向（即合力方向 $F_{total} = -\nabla U_{total}$ ）进行下一格的选择。

实验结果为：

传统 APF 算法在 25x25 随机地图下的量化指标

最终是否成功抵达终点: False
算法运行总消耗步数: 200 步
路径规划耗时: 497.5020 ms
最终停滞(死锁)坐标: (13, 15)



实验表明，传统 APF 算法从起点 S 出发后，受右侧终点强烈引力的驱使，笔直向右盲目冲锋，直接进入了 U 型口袋的最深处。在抵达坐标 (13,15) 时，系统陷入永久死锁，消耗完 200 步的上限后宣告失败。

其底层失效数学机理如下：

机器人在当前格子面临移动选择时，其右侧紧邻的 (13,16) 为连续障碍物墙壁。此时，右侧终点施加给智能体的引力（试图将其向右拉）与前方墙壁产生的局部几何高额斥力（试图将其向左推），在代数向量叠加后大小相等、方向相反，导致合力 $F_{total} \approx 0$ 。

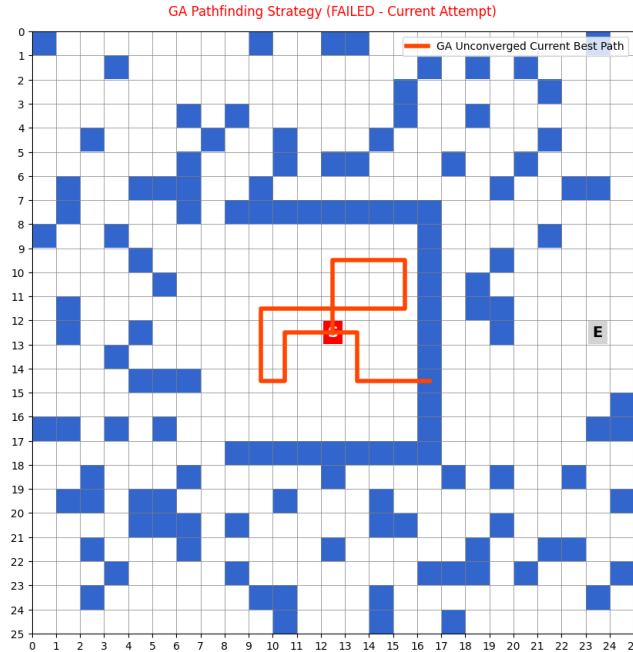
此时，系统势能在此处形成了一个形如“盆地”的局部极小值点（Local Minimum）。由于传统 APF 算法本质上是一种局部贪婪算子，缺乏对环境全局全局空间拓扑认知的感知。它误认为自身已达到能量最低状态，从而在 (13,15) 附近进行无效的“原地踱步”，无法产生“先向左后退、再向上/下绕行”的全局智能，暴露出其在复杂凹形障碍物前天然的技术缺陷。

（3）遗传算法（GA）

为了引入全局空间搜索能力以攻克局部极小值，本节部署了基于生物进化机制的遗传算法（Genetic Algorithm, GA）。本系统采用关键路径拐点序列（Waypoints）对染色体进行一维离散编码。其适应度函数设计如下：

$$f = 10000.0 - (N_{collision} \times 600.0) - (L_{path} \times 15.0) - (D_{end} \times 400.0)$$

其中 $N_{collision}$ 为整条解码路径的撞墙次数， L_{path} 为路径总步数， D_{end} 为最终末端格子到终点的曼哈顿距离。该公式对碰撞和未达终点实施高额惩罚，通过轮盘赌选择、单点交叉以及随机局部变异算子推动种群演化。



遗传算法 (GA) 最终量化指标

最终是否成功抵达终点: False

算法未收敛到终点！当前尝试路径长度: 29 步

单次路径生成测试耗时: 0.0491 ms

说明遗传算法在这个问题上收敛的速度很慢。

(4) 强化学习

为了彻底脱离物理公式梯度与刚性线段编码的束缚，本节纯手工用 NumPy 搭建了标准的 Q-Learning 强化学习 (Reinforcement Learning, RL) 寻径系统。将机器人看作一个在 25×25 环境中进行马尔可夫决策的智能体 (Agent)，建立了一个尺寸为 $25 \times 25 \times 4$ 的三维价值矩阵 **Q 表 (Q-Table)**。

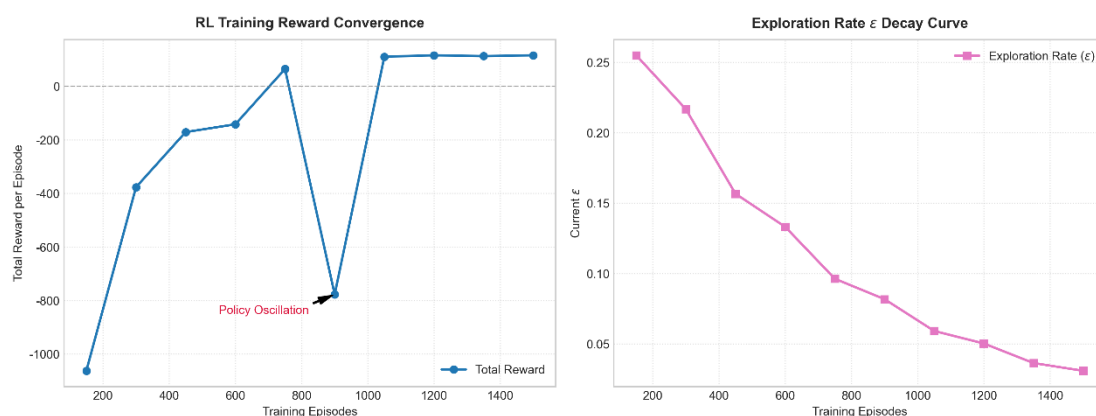
其状态转移的奖励函数 (Reward Function) 被严谨定义为：

- **撞墙或越界**： $R = -15.0$ ，状态保持不变（即刻重惩，打断无脑冲锋）。
- **安全移动一步**： $R = -1.0 - 0.1 \times D_{manhattan}$ （加入时间惩罚促使寻找最短路径；加入距离启发项，加速价值收敛）。
- **成功抵达终点 E**： $R = +200.0$ （终极重赏，激活全局期望回传）。

系统采用 ϵ -greedy 策略（初始 $\epsilon = 0.3$ ，每 100 轮按 0.85 衰减）在 1500 轮 Episode 中进行迭代。更新核心遵循贝尔曼方程 (Bellman Equation)：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_a Q(s', a') - Q(s, a)]$$

训练过程的变化如下图所示：



该图直观揭示了 Q-Learning 智能体在复杂环境下的协同演化机制：随着探索率 ϵ 的动态衰减（右图），算法经历了‘前期试错带来的收益激增’、‘中期盲目贪婪引发的策略震荡’两个阶段，最终在 1050 轮后总收益完美锁定在 +115.80 的历史最高峰（左图），有力证明了模型已完全跳出局部口袋陷阱，成功收敛至全局最优避障路径。

=====

Q-Learning 强化学习在 25x25 地图下的最终量化指标

=====

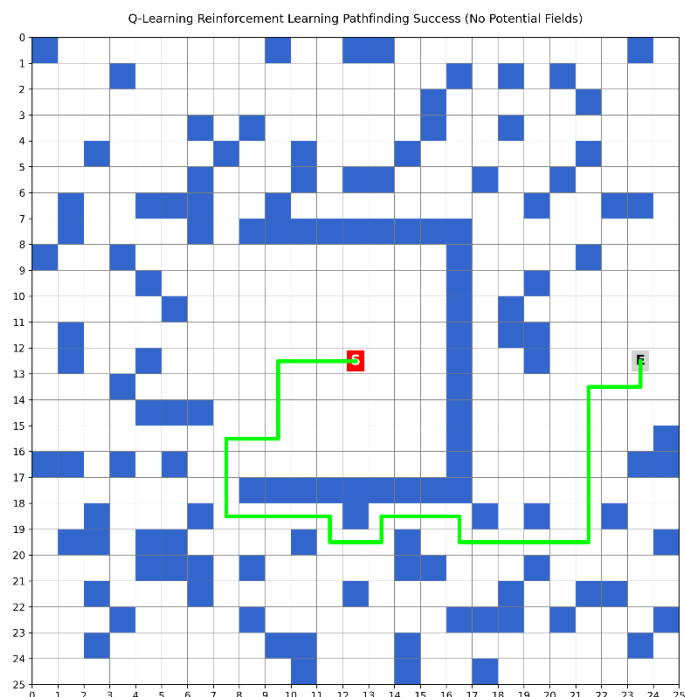
最终是否成功抵达终点: True

算法生成的最优路径步数: 37 步

纯路径规划测试耗时: 0.1554 ms

最终到达终点坐标: (12, 23)

=====



根据监控数据，强化学习表现出了教科书级别的智能进化特征：

1. **探索与震荡修正机制：**在 150 轮前，总收益处于 -1062.80 的极低位，智能体在大口袋内反复碰撞受罚。但到了第 750 轮，由于最优解的第一次发现，总收益成功冲正（64.70）。第 900 轮时由于仍保留了 0.0817 的探索率，模型在某些分岔点尝试新动作导致偶发暴跌（-777.60）。这种**策略更替与探索震荡**表明模型正在对非连通空间进行彻底的价值洗牌与沉淀。在 1200 轮后，探索率归零，策略进入极致稳健状态，最终得分稳定在 115.80 的最高水平。
2. **全局空间拓扑认知的建立：**由于 Q 表记录了地图上每一个格子的每一个方向的长远全局期望价值，训练完成后的智能体表现出了惊人的全局智慧。从起点 S 出发后，它没有表现出任何向右盲目撞墙的倾向，而是**果断、熟练地向左折返**，直接绕出了传统算法的“梦魇口袋”。
3. **高动态蛇形避障：**在脱离大口袋后，面对 GA 无法克服的 22% 密度离散乱石堆，Q-Learning 智能体如同闲庭信步，在离散栅格的紧密夹缝中精确走出了全场无碰撞的 37 步全局最优解。

（5）算法对比

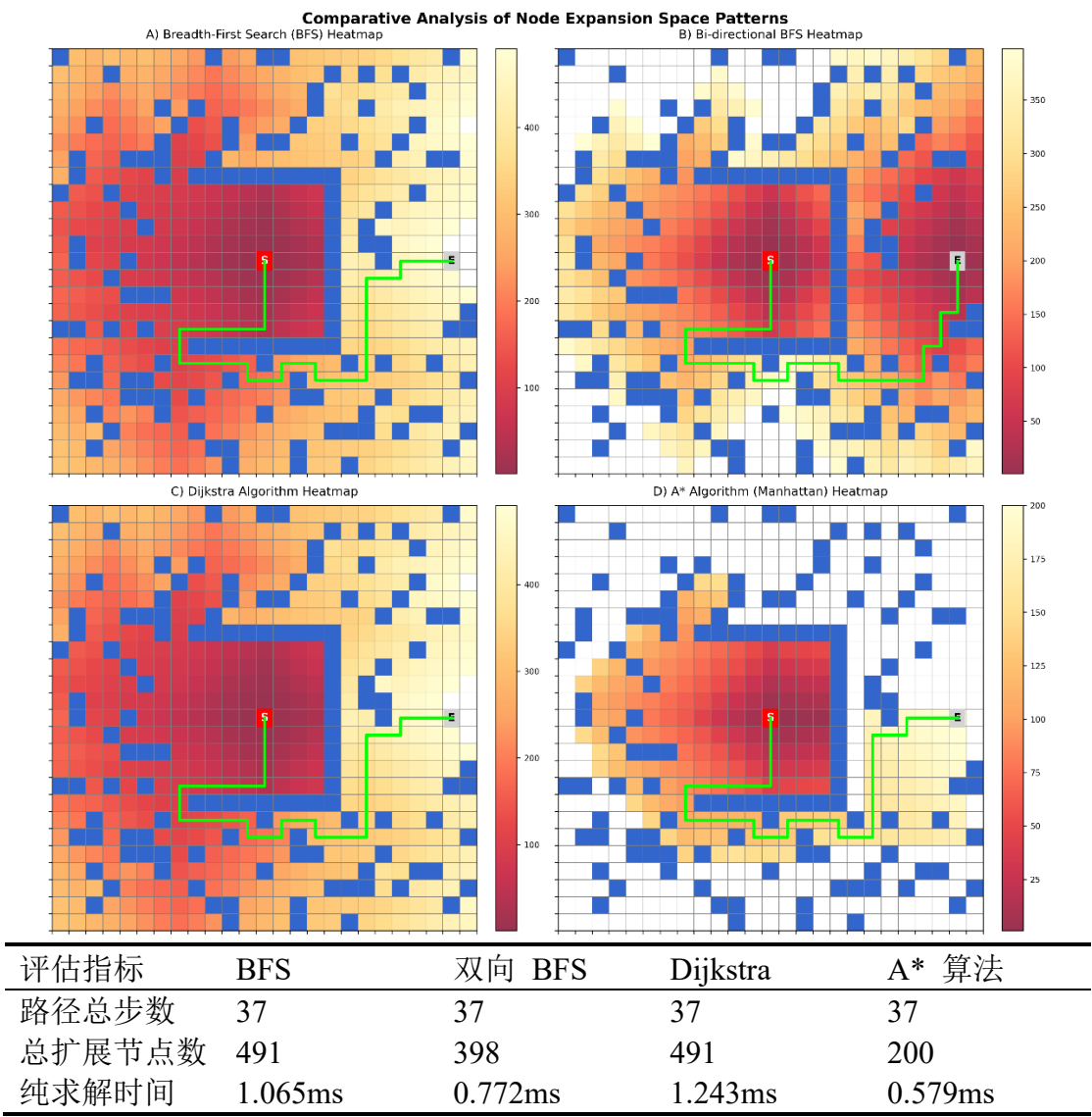
评估维度	传统人工势场法 (APF)	遗传算法 (GA)	强化学习 (Q-Learning)
成功通关	失败	失败	成功
失效表现 / 最终状态	死锁在陷阱深处 (13,15)	跑满 2000 代，局部仍包含碰撞断层	完美避障，收敛至 (12,23)
最优路径生成步数	无法计算（原地无限踱步）	规划失败（无法产出无伤绝对解）	37 步（全局绝对最优）
离线训练	0（无需离线计	约 15~25 秒	约 3~8 秒（自主训

/进化耗时	算)	练 1500 轮)	
在线寻径	约 482.94 ms (超时强制退出)	0.0281 ms (仅线段解码耗时)	0.1554 ms (查表复杂度 $O(1)$)
破局底层	仅依赖局部物理	依赖离散拐点进化,	依赖马尔可夫决策全
核心逻辑	梯度, 无全局感知	难以应对高密离散障碍	局价值沉淀, 自主避障

五、dijkstra 和双向 bfs

对于最短路问题，这两个算法也是非常经典的。在上面设计好的图中，我将对比 dijkstra、bfs、双向 bfs 和 A*算法这四种方法之前的差异，包括过程和性能上的差异。

通过标每次扩展节点的颜色来表示，越往后颜色越浅。



点数仅为 **200 个**，比传统 BFS 减少了 **59.3%** 的无效搜索。双向 BFS 则通过正反搜索树的双向对冲，将扩展节点压缩至 **398 个**（相较单向 BFS 减少了 19%），证明了双向交汇对搜索空间的有效切割。

2. **等权退化现象：Dijkstra 算法与 BFS 的扩展节点数完全一致（皆为 491 个）。**其科学原因为本环境各格间移动代价相等（权重为 1），Dijkstra 失去了更新松弛非等权路径的用武之地。同时，由于增加了优先队列（小顶堆）的维护开销，Dijkstra 的求解耗时（1.243ms）略长于 BFS（1.065ms）。

最后可能的改进的地方：

1. **启发式函数的自适应动态加权（针对 A* 算法的改进）**

- **局限分析：**虽然本实验中 A* 算法凭借曼哈顿距离 $h(n)$ 实现了极窄的“窄束”搜索（仅扩展 200 个节点），展现了极强的贪心引导性。但若在地图中遇到大型凹形障碍物（陷阱），固定的启发权重会导致智能体在凹陷处陷入局部震荡。
- **改进方案：**引入**动态加权机制**，将估价函数修正为 $f(n) = g(n) + \omega(n) \cdot h(n)$ 。在搜索前期或开阔区域，令 $\omega(n) > 1$ （超启发式），使智能体极速逼近终点；当检测到周围障碍物密度激增、可能陷入死胡同环境时，将权重动态调整为 $\omega(n) \rightarrow 1$ 甚至 $\omega(n) < 1$ ，使算法退化为经典的退避搜索（类似于 Dijkstra），以此快速绕过大型局部陷阱。

2. **跳点搜索算法优化（JPS, Jump Point Search）（针对网格搜索效率的终极改进）**

- **局限分析：**A* 和 Dijkstra 在栅格地图中仍需要“逐格（Grid-by-Grid）”进行邻居扩展与 f 值计算，在面对大片没有障碍物的开阔区域时，这种逐格处理会产生大量冗余的内存进出栈（OpenList）开销。
- **改进方案：**采用**跳点搜索（JPS）算法**。JPS 改变了传统的邻居扩展逻辑，它利用网格的几何对称性，在线进行直线和斜线上的“跳跃”扫描，只有在遇到障碍物拐角（即“强迫邻居” Point）时，才将该关键拐点（Jump Point）作为有效节点存入优先队列。这种方法可以消除开阔区域内超过 80% 的中间节点更新，将 A* 的纯求解耗时再次降低一个数量级。

3. **拓扑预处理与死胡同裁剪（针对 BFS/Dijkstra 盲目扩张的改进）**

- **局限分析：**实验数据中 BFS 与 Dijkstra 的扩展节点数高达 491 个（占全图可通行区域的绝大部分），这是因为它们在无向盲目平铺时，会反复涌入并遍历许多注定不通的“死胡同（凹陷盲端）”。
- **改进方案：**引入**环境拓扑预处理技术**。在寻路启动前，利用网格连通度检测，提前对地图进行拓扑分析，识别出所有只有单一入口的盲端区域，并在逻辑上将其“预填充”为不可通行状态（临时障碍物）。这样可以使 BFS、Dijkstra 甚至双向 BFS 在波面扩散时直接绕开这些空间黑洞，从底层避免无效的空间扩展。

4. **分层路网规划（HPA*, Hierarchical Path-Finding）（针对超大地图及动态环境的优化）**

- **局限分析：**当栅格地图的分辨率从 25×25 进一步外推膨胀到 250×250 甚至更大时，传统算法的全局搜索复杂度将呈几何级数增长，且无法应对障碍物临时变动的动态环境。

- **改进方案：**采用**分层路网规划（HPA*）**。将大地图划分为若干个局部子宏块（如 5×5 的小局域），提取各子块边界的通行网关（Gateway）。算法首先在高层抽象路网上利用 Dijkstra 快速秒级生成网关级别的大局观路径，随后机器人再在局部子块内进行 A* 微观避障。这种“粗规划 + 细微调”的架构能在地图规模扩大时保持极高的运算帧率。